Load Balancer Simulator: complexity analysis

Costs of the code as written (`lbsim/algorithms.py`, `lbsim/health.py`, `lbsim/simulator.py`). **n** = servers, **h** = healthy servers (the heap holds only these, so h ≤ n), **m** = requests in flight, **R** = requests in a run. Every balancer call takes the balancer's lock: O(1) when uncontended; contended calls run one at a time.

1. Shared interface

```
acquire():                      release(s):                     set_healthy(s, up):
    lock                            lock                            lock
    s = pick()                      if s.connections == 0: error    s.healthy = up
    if s: s.connections += 1        s.connections -= 1              on_health_change(s)
          on_load_change(s)         on_load_change(s)               unlock
    unlock; return s                unlock
```

Each algorithm supplies `pick`, `on_load_change` and `on_health_change`. `acquire` is what a request calls: pick, then count the connection.

2. The heap (`MinHeap`)

An array of `(key, id)` pairs kept as a binary min-heap (ties broken by id), plus `pos`: id → array index.

```
sift_up(i):    while i > 0 and a[i] < a[(i-1)//2]:  swap(i, (i-1)//2);  i = (i-1)//2
sift_down(i):  loop:  m = index of the smallest of a[i], a[2i+1], a[2i+2]
                      if m == i: stop
                      swap(i, m);  i = m
swap(i, j):    exchange a[i], a[j] and fix pos[a[i].id], pos[a[j].id]                    # O(1)

push(id, key):    a.append((key, id));  pos[id] = len(a) - 1;  sift_up(len(a) - 1)
update(id, key):  i = pos[id];  old = a[i];  a[i] = (key, id)
                  if a[i] < old: sift_up(i)  else: sift_down(i)
remove(id):       i = pos.pop(id);  last = a.pop()
                  if i < len(a):  a[i] = last;  pos[last.id] = i;  sift_up(i);  sift_down(i)
peek():           a[0] if a else none
```

Operation	Time	As implemented
`peek`, membership (`id in heap`)	O(1)	array read; dict lookup
`push`	O(log h)	one sift-up
`update`	O(log h)	one sift: up if the key fell, down otherwise. This is decrease-key and increase-key, found in O(1) through `pos`
`remove`	O(log h)	refill the hole from the last slot, then sift up and down; at most one of the two moves it
Build (`h` pushes, plus the id → server map)	O(n + h log h)	the code pushes one at a time, not with a linear-time heapify
Space	O(n)	h array entries + h map entries, plus the balancer's map of all n servers

3. The three algorithms

```
Round Robin
pick():  for i in 0 .. n-1:
             s = servers[(next + i) mod n]
             if s.healthy:  next = (next + i + 1) mod n;  return s
         return none
on_load_change(s), on_health_change(s):  nothing

Least Connections / Weighted Least Connections
score(s) = s.connections                  # Least Connections
score(s) = s.connections / s.weight       # Weighted Least Connections

init:                for s in servers: if s.healthy: heap.push(s.id, score(s))
pick():              top = heap.peek();  return server[top.id] if top else none
on_load_change(s):   if s in heap: heap.update(s.id, score(s))
on_health_change(s): if not s.healthy and s in heap: heap.remove(s.id)
                     elif s.healthy and s not in heap: heap.push(s.id, score(s))
```

	`pick`	`acquire`	`release`	`set_healthy`	**Space**
Round Robin	O(1); O(d + 1) when it skips d down servers, so O(n) worst case	same as `pick`	O(1)	O(1)	O(1) beyond the O(n) server list
Least Connections, Weighted	**O(1)** (peek and a dict lookup)	**O(log h)**: the pick, then an `update` after the connection is counted (the key rose, so it sifts down)	O(log h): `update`, the key fell, sifts up	O(log h): `remove` when marked down, `push` when marked up	O(n)
Least Connections by scanning (the baseline in `docs/bench.py`)	O(n): it visits every server to test its health	O(n)	O(1)	O(1)	O(n)

So the logarithm in a heap pick lives in the update that follows the choice, not in the choice itself. The choice is O(1) because the least-loaded server is always the root.

4. Health checks

```
check(now):                                  # one probe round, under the checker's lock
    for s in servers:
        if s.alive != s.healthy:
            balancer.set_healthy(s, s.alive);  log(now, s.id, s.alive)

note_refused(s, now):                        # passive check, only when passive_after = N > 0
    refused[s] += 1
    if refused[s] >= N and s.healthy:
        refused[s] = 0;  balancer.set_healthy(s, false);  log(now, s.id, false)

note_ok(s):  refused[s] = 0
```

	Time	**Space**
One probe round	O(n) for the scan, plus one `set_healthy` per server whose state changed: O(log h) each with a heap balancer, O(1) with Round Robin	O(1)
A run	about T / I rounds (T = time traffic keeps arriving, I = check interval; probes stop at the last arrival): O(n T / I + c log h) for c changes	O(c) for the log
`note_refused`, `note_ok`	O(1); the call that crosses N also pays one `set_healthy`, O(log h)	O(n) counters

5. A request in the simulator

- **Accepted:** an `acquire` (O(log h)), a push onto the in-flight heap (O(log m)), and later a pop (O(log m)) and a `release` (O(log h)).
- **Refused or overloaded:** no completion event. It releases at once inside its arrival event: `acquire` plus `release`, still O(log h).
- **No healthy server:** a heap balancer returns at once, O(1); Round Robin first scans past all n down servers, O(n).
- **Choosing the next event** compares the heads of four queues: O(1).

So time is O(log h + log m) per request and O(R (log n + log m)) for a run, plus the health checks above. With our fleet m ≤ 78 (the sum of the connection limits, 3 x the weights). Space is dominated by the R records, O(R); the in-flight heap adds O(m) and the fleet O(n). Computing the metrics afterwards is O(R) per table.

6. What was measured (`docs/bench.py`, `lbsim/facts.py`)

One balancer operation (acquire or release) on n servers with n connections open. macOS-27.2-arm64-arm-64bit-Mach-O, Python 3.14.3, best of three; read the shape, not the microseconds.

servers	heap, µs per operation	linear scan, µs per operation	scan ÷ heap
5	0.74	0.50	0.68
15	0.95	0.88	0.93
20	1.06	1.09	1.03
50	1.27	2.22	1.74
500	1.95	19.13	9.81
5,000	2.89	185.87	64.39
50,000	6.00	1862.53	310.22

- **At our 5 servers the scan is faster:** 0.50 µs against the heap's 0.74 µs. The heap's sifting and bookkeeping cost more than looking at five numbers.
- **They cross between 15 and 20 servers** (scan ÷ heap is 0.93 at 15, 1.03 at 20). At 50,000 servers the heap is **310 times faster**.
- **The heap grows logarithmically, the scan linearly.** The heap's cost rises 8.1-fold for 10,000 times the servers (a logarithm rises 6.7-fold over that range); the scan's rises 3,725-fold.
- **A whole simulation costs a flat 2.2 to 2.3 µs per request for Round Robin and 3.2 to 3.4 µs for the heap variants,** from 951 to 99,855 requests, as O(R (log n + log m)) with small n and m predicts.

7. Where this differs from REPORT.md section 3

REPORT.md's conclusions stand, but its shorthand is looser than the code in four places:

1. **log h, not log n.** The heap holds only healthy servers, so its operations are O(log h). REPORT's O(log n) is a valid but looser bound.
2. **"Pick" bundles two costs.** REPORT's row "Pick: O(1) read + O(log n)" is `acquire`. `pick()` alone is O(1); the O(log h) is the `update` that counts the connection.
3. **Not every request has a completion event.** REPORT says each request is one arrival and one completion. Only accepted requests have a completion; a failed request releases inside its arrival event. The order of growth is unchanged.
4. **Health checks have two parts, and the passive check was not costed.** A probe round is an O(n) scan plus O(log h) per state change; the passive check is O(1) per request and O(log h) when it fires. Round Robin's `release` and `set_healthy` are O(1).